

DeepSeek Harness UI Plugin Book

DeepSeek Harness contributors

Learning path

1. Understand the UI architecture
2. Try the idea in chat first
3. Plan the production demo
4. Create the package manifest
5. Implement the Host Tool
6. Build the Studio component
7. Register the keyed Tool Slot
8. Mount and install the plugin
9. Ask the agent to use the component
10. Use the same capability from Copilot or ACP
11. Build an external UI with the SDK
12. Test the component and lifecycle
13. Verify the installed result
14. Choose the right UI pattern
15. Continue with production references

Learning path

Build one agent-driven feature and carry it through four forms: a temporary UI created in chat, a production DSH Studio plugin, an ACP client such as an editor integration, and a custom external application. The finished production demo adds a model-facing `study_progress` tool and a dedicated Studio card for its calls.

You will learn how to:

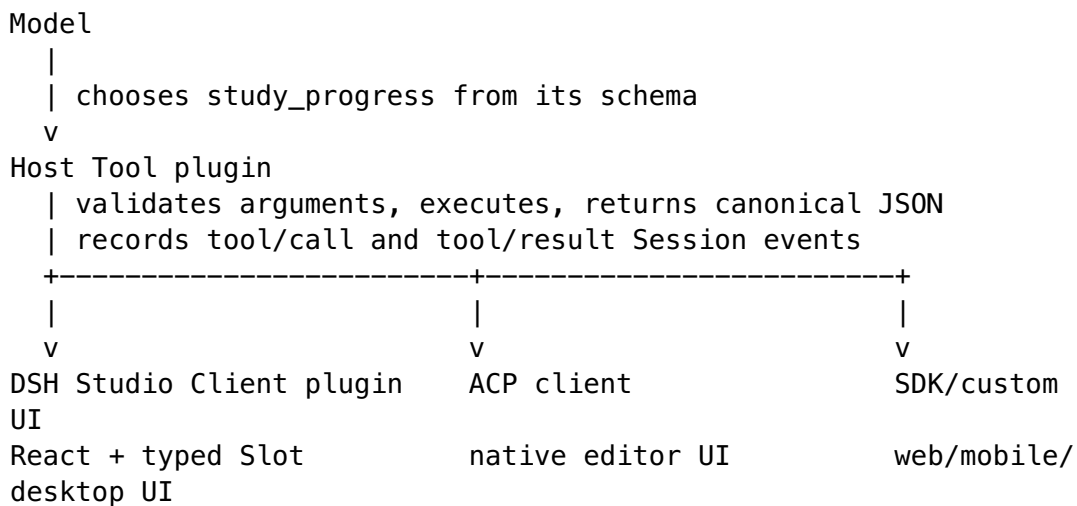
- Choose between a temporary dynamic UI and an installed plugin
- Build a React component that follows the DSH Client Slot rules
- Connect an agent Tool to a replayable UI card
- Add the Client plugin to DSH Studio
- Expose the same agent capability to Copilot or another ACP client
- Drive the agent from TypeScript, Python, or another external UI
- Test component behavior, plugin lifecycle, assembled browser output, and installation

Complete [Your first Harness plugin](#) before starting. Use Node.js ^22.19 or >=24, pnpm, and a local checkout with `pnpm install` complete.

Studio React components do not cross process boundaries. Studio renders a Client plugin. ACP and SDK clients receive protocol data and render their own native components. The shared unit is the Tool and its durable Session events, not the React tree.

1. Understand the UI architecture

DSH separates agent behavior from presentation:



The Host Tool owns behavior and model-facing data. A Studio Client plugin owns rendering. An external client owns its own UI and consumes ACP or SDK events. This split keeps Tool behavior available when no browser is open and keeps Session replay independent of one UI framework.

Use these extension points:

Goal	Extension point
Show a standard Tool card	<code>presentCall</code> and <code>presentResult</code> render intent
Replace one Tool's Studio card	keyed <code>tool.call.toolview</code> Slot
Add a durable business row to chat	<code>ConversationNodeDefinition</code> and keyed <code>conversation.chat.node</code> Slot
Add settings or sidebar UI	the matching declared Slot
Let the model act	a Host Tool registered on <code>ctx.tools</code>
Let Client UI call the Host	a Remote API or package-private dynamic Cordis handler
Drive DSH from an editor	ACP over JSON-RPC stdio
Drive DSH from an application	TypeScript or Python SDK over JSON-RPC stdio

Read [Tool authoring](#) for the complete Tool contract and [Add a conversation node](#) when the UI represents a durable multi-event workflow rather than one Tool call.

2. Try the idea in chat first

Dynamic Cordis plugins are useful for a disposable prototype in the current DSH process. Start Studio:

```
pnpm dsh web --no-open
```

Open the printed URL, start a session, and send this prompt:

```
Create a temporary Study Progress UI in this DSH session. It
should show a subject,
completed lessons, total lessons, and a progress bar. Use a Client
dynamic Cordis
plugin, inspect the available Slots before choosing where to mount
it, and ask for
browser approval when required.
```

Review the generated code in the `cordis_define` card. Approve the Client code only when you trust it. After the run succeeds, refer to the plugin with its returned identifier:

```
@study-1 add a Reset action and make the empty state clear.
```

The actual identifier may differ. The `@pluginId` reference tells the agent to inspect the existing package, append an immutable version, and run or update it.

Dynamic definitions live only in process memory. A DSH restart invalidates their `pluginId` and `packageId`. Recreate the prototype as a new dynamic plugin or continue with the installed-plugin lab below when it must survive restarts.

3. Plan the production demo

The production demo uses one dual-face package:

```
dsh-study-progress/
  package.json
  cordis.patch.yml
  tsconfig.json
  tsdown.config.ts
  src/
    index.ts
    invariant.ts
    client/
      index.ts
      StudyProgressCard.tsx
      StudyProgressCard.module.css
      css-modules.d.ts
  tests/
```

```
browser-plugin.client.spec.ts
study-progress-card.client.spec.tsx
```

The Node entry registers `study_progress`. The browser entry registers a keyed Studio card. The bundle patch mounts the Host and Client rows. Keeping both halves in one package is appropriate because they ship as one feature; larger capabilities may split the Tool, service, and UI into independently versioned packages.

The Tool result should be useful without Studio:

```
{
  "subject": "Cordis lifecycle",
  "completed": 3,
  "total": 5,
  "percent": 60,
  "message": "3 of 5 lessons complete"
}
```

Studio may turn that data into a progress card. ACP may show the rendered text in an editor card. An external dashboard may draw its own progress bar.

4. Create the package manifest

Create the package outside this repository when you want an independently installable plugin. During repository development, place an official package under the appropriate `packages/<group>/<name>/` directory and follow [Add a workspace package](#).

Use this standalone `package.json` as the starting point. Replace workspace ranges with the published DSH version you target before distributing it.

```
{
  "name": "dsh-study-progress",
  "version": "0.1.0",
  "type": "module",
  "main": "lib/index.js",
  "types": "lib/types/index.d.ts",
  "exports": {
    ".": {
      "types": "./lib/types/index.d.ts",
      "default": "./lib/index.js"
    },
    "./client": {
      "types": "./lib/types/client/index.d.ts",
      "default": "./lib/client.js"
    },
    "./invariant": {
      "types": "./lib/types/invariant.d.ts",
      "default": "./lib/invariant.js"
    },
    "./cordis.patch.yml": "./cordis.patch.yml"
  }
}
```

```

},
"files": [
  "lib/index.js",
  "lib/client.js",
  "lib/invariant.js",
  "lib/types/**/*.d.ts",
  "cordis.patch.yml"
],
"scripts": {
  "build": "tsc -b && tsdown",
  "prepare": "pnpm run build",
  "test": "vitest run"
},
"dsh": {
  "bundle": { "patch": "./cordis.patch.yml" },
  "client": {
    "platform": "web",
    "inject": ["@deepseek-ai/dsh-client-ui-tool"]
  }
}
}

```

The `dsh.bundle` entry makes the package installable into a profile. The `dsh.client` entry tells the Web module loader that `./client` is a browser plugin. Client packages also need the Cordis, React, DSH Tool, Client runtime, Slot, primitive, and invariant dependencies used by their source. Follow the exact peer/development dependency rules in [the Client package checklist](#).

5. Implement the Host Tool

The Host Tool accepts completed and total lesson counts, computes a stable result, and emits model-facing text independently from Studio.

```

import type { Context } from '@deepseek-ai/cordis'
import { defineTool } from '@deepseek-ai/dsh-tools'

export const name = 'study-progress'
export const inject = ['tools']

export function apply(ctx: Context): void {
  ctx.tools.register(defineTool({
    name: 'study_progress',
    description: 'Report a learner progress checkpoint for one subject.',
    parameters: {
      subject: { type: 'string', required: true, description: 'Subject being studied.' },
      completed: { type: 'number', required: true, description: 'Completed lesson count.' },
      total: { type: 'number', required: true, description: 'Total lesson count.' },
    },
  }},

```

```

output: {
  schema: {
    type: 'object',
    additionalProperties: false,
    properties: {
      subject: { type: 'string', required: true },
      completed: { type: 'number', required: true },
      total: { type: 'number', required: true },
      percent: { type: 'number', required: true },
      message: { type: 'string', required: true },
    },
  },
  render: (_args, value) => [{ type: 'text', text:
value.message }],
  presentationMeta: (_args, value) => value,
},
execute(args) {
  if (!Number.isInteger(args.completed) || \!
Number.isInteger(args.total)) {
    throw new Error('completed and total must be integers')
  }
  if (args.total <= 0 || args.completed < 0 || args.completed
> args.total) {
    throw new Error('expected 0 <= completed <= total and
total > 0')
  }
  const percent = Math.round((args.completed / args.total) *
100)
  return Promise.resolve({
    ...args,
    percent,
    message: `${args.completed} of ${args.total} lessons
complete`,
  })
},
}))
}

```

The schema validates JSON types before execute. The explicit checks enforce relationships that the schema cannot express. `presentationMeta` stores the replayable fields needed by a card. Do not read files, clocks, Session state, or random values in a presenter or card model: replay must produce the same view from durable call and result data.

6. Build the Studio component

The component receives `ToolCallViewProps`. It does not receive `ctx`, import a Host service, or subscribe directly to business data. Normalize the frozen Tool block in a pure helper, then render the result.

```

import type { ToolCallViewProps } from '@deepseek-ai/dsh-client-
ui-tool/client'

```

```

import css from './StudyProgressCard.module.css'

interface ProgressCardModel {
  subject: string
  completed: number
  total: number
  percent: number
  state: 'running' | 'complete' | 'error'
  message?: string
}

function readObject(value: unknown): Record<string, unknown> |
null {
  return typeof value === 'object' && value !== null
    ? value as Record<string, unknown>
    : null
}

function modelOf(block: ToolCallViewProps['block']):
ProgressCardModel {
  const settled = 'kind' in block
  const argsRaw = (settled ? block.call?.argsRaw :
block.argsRaw) ?? '{}'
  let args: Record<string, unknown> = {}
  try { args = readObject(JSON.parse(argsRaw)) ?? {} } catch { /*
streaming JSON may be incomplete */ }
  const meta = settled ? readObject(block.meta) : null
  const source = meta ?? args
  return {
    subject: typeof source.subject === 'string' ? source.subject :
'Study progress',
    completed: typeof source.completed === 'number' ?
source.completed : 0,
    total: typeof source.total === 'number' ? source.total : 0,
    percent: typeof source.percent === 'number' ? source.percent :
0,
    state: !settled ? 'running' : block.isError ? 'error' :
'complete',
    message: settled && block.isError ? 'The progress update
failed.' : undefined,
  }
}

export function StudyProgressCard({ block }: ToolCallViewProps) {
  const model = modelOf(block)
  return (
    <section className={css.card} aria-label="Study progress">
      <div className={css.heading}>
        <strong>{model.subject}</strong>
        <span>{model.state === 'running' ? 'Updating' : `
${model.percent}%`}</span>
      </div>
      <progress max={100} value={model.percent} aria-label={`
${model.percent}% complete`} />
    </section>
  )
}

```

```

      <div className={css.detail}>{model.completed} of
{model.total} lessons</div>
      {model.message !== undefined && <div
role="alert">{model.message}</div>}
    </section>
  )
}

```

Use a CSS Module and shared `--dsw-*` tokens. Do not use literal product colors, global selectors, Tailwind, or a second component library. Keep product copy localized when the plugin ships to users; the compact example keeps English text visible so the data flow is easy to follow.

7. Register the keyed Tool Slot

The `tool.call.toolview` Slot dispatches by exact Tool name. Registering key: `'study_progress'` replaces the generic Studio row only for this Tool.

```

import type { ClientContext } from '@deepseek-ai/dsh-client-
runtime/client'
import type {} from '@deepseek-ai/dsh-client-ui-tool/client'
import { StudyProgressCard } from './StudyProgressCard.tsx'

export const inject = ['slots']

export function apply(ctx: ClientContext): void {
  ctx.slots.inject('tool.call.toolview', () =>
ctx.slots.register({
  name: 'tool.call.toolview',
  key: 'study_progress',
}, StudyProgressCard))
}

```

`ctx.slots.inject` waits for the Slot declaration, registers while its owner exists, unregisters when that declaration disappears, and registers again after redeclaration. Do not replace it with a bare `slots.register` into a Slot owned by another package.

Use the four derived prop shares for more advanced components:

- `PropsRuntime` for owner data and standard session hooks
- `PropsRenderSlots` for child Slots declared by this entry
- `PropsStore` for shared interaction state that survives remounts
- `InjectFace` for plain callbacks, values, and framework-bound observable hooks

Component-local state stays in React. Session business data stays in the Client runtime. Shared view state uses an entry-declared store factory. Components never receive `ctx`.

8. Mount and install the plugin

Create `cordis.patch.yml` with Host and Client rows:

```
- insert:
  - id: study-progress
    name: dsh-study-progress

  - id: ui-study-progress
    name: dsh-study-progress/client
```

For an in-repository official Client package, add its package name to the Web bundle dependencies and add a `dsh.client` row to `packages/bundle/web-app/cordis.patch.yml`. Add the package to `tsconfig.client.json` as described by [the workspace package checklist](#).

For a standalone package, build and install it into a profile:

```
pnpm run build
dsh plugin --profile study add ./dsh-study-progress
dsh --profile study --dump-config
dsh --profile study web --no-open
```

The dump must contain a `dsh-study-progress` bundle layer and both rows. Remove it with:

```
dsh plugin --profile study remove dsh-study-progress
```

Git installations need a self-contained prepare script. pnpm 10 requires the user to allow that package's build in the profile's `pnpm-workspace.yaml`. Prefer a registry package or tarball containing built artifacts when install-time code execution is undesirable. [Package and install a plugin](#) owns the complete profile, bundle, Git, and trust workflow.

9. Ask the agent to use the component

The agent never clicks the React component to make progress appear. It calls the Host Tool; Studio observes the resulting durable Tool events and selects the keyed renderer.

Start a session with the plugin installed and send:

```
Create a five-part study plan for Cordis lifecycle. After listing
the plan, call
study_progress for "Cordis lifecycle" with completed=1 and
total=5. When I tell
you that I completed another lesson, call the tool again with the
new count.
```

Then send:

```
I completed the next lesson.
```

Verify these outcomes:

1. The model calls `study_progress` with structured arguments.
2. The Host validates and records the call and result.
3. Studio selects `StudyProgressCard` for the exact Tool name.
4. Reloading the page reconstructs the same completed card from the Session log.
5. A client without the custom component still receives 3 of 5 lessons complete from `output.render`.

Use a Host Tool or Remote method for UI actions that must change agent state. A button callback calls that typed operation with plain JSON; the Host records any model-visible consequence as a Session event. Never mutate Host state only inside React.

10. Use the same capability from Copilot or ACP

ACP is the editor-facing automation transport. Run the checked-in ACP composition:

```
pnpm --dir . run demo:acp
```

Configure an ACP-compatible client to launch that command over stdio, then use `initialize`, `session/new`, and `session/prompt`. Permission requests arrive through `session/request_permission`; committed assistant text and images arrive through `session/update`.

ACP intentionally does not carry Studio React components, transcript replay, reasoning, plans, titles, or Tool presentation. An editor such as Copilot renders its own controls from protocol capabilities. To expose `study_progress`, add its Host package to the ACP composition used by that client. The client will receive the agent's committed answer; richer Tool activity requires a transport that projects the corresponding Session events.

Treat the ACP package contract as authoritative. Do not advertise a Studio-only interaction as portable to ACP.

11. Build an external UI with the SDK

The TypeScript and Python SDKs launch a DSH runtime over JSON-RPC stdio and return root Session events. Your application maps those events to its own component state.

Python installation and first call:

```
python -m pip install deepseek-harness-sdk
```

```
from deepseek_harness import DeepSeekHarness
```

```
with DeepSeekHarness(  
    provider="deepseek-official",  
    model="deepseek-v4-flash",  
    cordis="/absolute/path/to/study-agent.cordis.yml",  
):
```

```

) as harness:
    result = harness.run(
        "Build a five-part Cordis plan and report one completed lesson.",
        session_id="student-001",
    )

print(result.final_response)
for event in result.events:
    # Route tool/call and tool/result events into application-
    # owned view models.
    print(event)

```

TypeScript applications use DeepSeekHarness from @deepseek-ai/dsh-sdk-client with an explicit runtime launch command. Subscribe with the lower-level HarnessClient when the UI needs notifications while work is in progress. Close the harness so the runtime subprocess is reaped.

An external UI should:

1. Start or connect to a trusted DSH runtime.
2. Create or select an application-owned session.
3. Send a prompt.
4. Reduce ordered Session events into plain application state.
5. Render that state with the application's native component system.
6. Send typed commands or new prompts for user actions.
7. Preserve the Session id only when the next interaction should continue that conversation.

Do not parse the assistant's prose to recover identifiers or progress. Prefer canonical Tool values, durable metadata, or dedicated Session events. [The TypeScript SDK](#), [Python SDK](#), and [JSON-RPC example](#) define the supported process lifecycle and event access.

12. Test the component and lifecycle

Use four test levels. Each proves a different property.

Component test

Render StudyProgressCard with realistic running, successful, malformed-streaming, and failed blocks. Assert accessible text, progress values, and actions. Do not assert CSS class names, render counts, or Slot internals. Put `// @vitest-environment jsdom` on the first line of a component spec.

```

// @vitest-environment jsdom
import { render, screen } from '@testing-library/react'
import { describe, expect, it } from 'vitest'
import { StudyProgressCard } from '../src/client/StudyProgressCard.tsx'

describe('StudyProgressCard', () => {

```

```

    it('renders a completed checkpoint from durable metadata', () =>
    {
      render(<StudyProgressCard {...completedToolProps} />)
      expect(screen.getByText('Cordis lifecycle')).toBeTruthy()

      expect(screen.getByRole('progressbar')).toHaveAttribute('value',
        '60')
      expect(screen.getByText('3 of 5 lessons')).toBeTruthy()
    })
  })
})

```

Keep the fixture builder in the test package; do not widen the plugin's public exports for tests.

Plugin wiring test

Create a real Cordis Context, mount SlotRegistry, declare `tool.call.toolview`, mount the Client plugin, and assert that the `study_progress` entry appears. Dispose the plugin fiber and assert that the entry disappears. This is the HMR-safety proof required for registry contributions.

Use [ui-user-questions browser plugin tests](#) as the compact reference and [ui-cordis](#) as a multi-Slot production example.

Host Tool test

Execute through the real Tool registry. Cover valid progress, zero or negative totals, completed counts above total, cancellation if execution becomes asynchronous, canonical output, model rendering, and presentation metadata. Assert behavior rather than calling `execute` directly.

Assembled browser test

A product-visible UI plugin needs a real Loader composition test and a browser snapshot. Boot the actual package through a test `cordis.yml`; mock only the model or other nondeterministic boundary. Verify that the Tool call produces the expected visible card and that replay reconstructs it.

Run the narrow checks while developing:

```

pnpm --filter dsh-study-progress test
pnpm --filter dsh-study-progress bundle
pnpm run test:gui
DSH_SNAPSHOT=replay pnpm run test:web

```

For an official in-repository package, also run the selected typecheck, lint, package constraints, documentation checks, and pre-push checks required by the changed files. The [testing policy](#) defines when unit, coverage, real-API, snapshot, and browser tests are required.

13. Verify the installed result

Use this acceptance checklist before publishing:

1. `dsh --profile study --dump-config` shows the bundle and both plugin rows.
2. The Client bundle exists as `lib/client.js`.
3. Studio starts without Loader or module-graph errors.
4. The model schema contains `study_progress`.
5. A Tool call renders the custom card in Studio.
6. The same call has useful text in a client without the custom renderer.
7. Page reload reconstructs the settled card.
8. Plugin disposal removes the Tool and Slot entry.
9. Reinstalling the package into a clean profile works from the distributed tarball or registry artifact.
10. ACP and SDK examples describe only the data their protocols actually expose.

Inspect the packed package rather than trusting the source tree:

```
pnpm pack
tar -tf dsh-study-progress-0.1.0.tgz
dsh plugin --profile clean-study add ./dsh-study-
progress-0.1.0.tgz
dsh --profile clean-study --dump-config
```

The archive must contain every exported runtime file and no source-only secrets, credentials, test fixtures, or stale build output.

14. Choose the right UI pattern

Requirement	Recommended pattern
Prototype visible behavior during one Studio process	Dynamic Cordis Client plugin
Render one Tool call specially in Studio	keyed <code>tool.call.toolview</code> entry
Render a durable workflow independent of one Tool call	Conversation Node definition and keyed renderer
Add a user preference	settings section plus settings card
Add navigation or a global action	declared sidebar or layout Slot
Support an ACP editor	Host capability plus ACP-native client UI
Support a standalone application	SDK Session events plus application-native UI
Share one exact React component across Studio and another app	Extract a framework-neutral library only when both build systems can own it; do not cross the DSH process boundary with React nodes.

15. Continue with production references

Use these documents as the source of truth after completing the lab:

- [Web Client package rules](#) for Slot, props, state, dependency, styling, and test constraints
- [Web styling](#) for tokens and CSS Modules
- [Tool authoring](#) for schemas, execution, model rendering, durable metadata, and neutral UI intents
- [Conversation Nodes](#) for replayable business rows
- [Package installation](#) for bundles and profiles
- [Architecture](#) for plugin composition and durable Session events
- [ACP](#) for editor automation limits
- [TypeScript SDK](#) and [Python SDK](#) for external application integration

The core design rule is consistent across every target: keep agent behavior and durable facts on the Host, keep presentation in the client that owns it, and make every UI reconstructable from typed inputs rather than hidden live state.